services\analysis\trend_analyzer.py

```
from typing import Dict, List, Any
from datetime import datetime, timezone, timedelta
from collections import defaultdict
from services.data.api_client import api_client

def get_cve_trends_last_30_days() -> Dict[str, Any]:
"""Get CVE trends for last 30 days - API ONLY"""
print("[CVE Trends] Fetching 30-day trends from API...")

# Get recent vulnerability data from API
cves = api_client.get_cves_last_30_days()

# Initialize daily counting structure
daily_counts = defaultdict(int)
today = datetime.now(timezone.utc).date()

# Pre-fill last 30 days with zero counts
for i in range(30):
day = today - timedelta(days=29-i)
daily_counts[day.strftime('%Y-%m-%d')] = 0

# Process each CVE to count by publication date
for cve in cves:
pub_date = cve.get('Published', '')
if pub_date:
try:
if 'T' in pub_date:
dt = datetime.fromisoformat(pub_date.replace('Z', '+00:00'))
else:
dt = datetime.strptime(pub_date[:10], '%Y-%m-%d')

date_key = dt.date().strftime('%Y-%m-%d')
if date_key in daily_counts:
daily_counts[date_key] += 1
except:
continue

# Build chart-ready data structure
sorted_dates = sorted(daily_counts.keys())
result = {
'labels': sorted_dates,
'values': [daily_counts[date] for date in sorted_dates],
'total_cves': len(cves),
'date_range': {
'start': sorted_dates[0] if sorted_dates else '',
'end': sorted_dates[-1] if sorted_dates else ''
}
}

print(f"[CVE Trends] Generated trends: {result['total_cves']} CVEs over
{len(sorted_dates)} days")
return result

def get_vulnerabilities_over_time_last_n_years(years=1) -> Dict[str, Any]:
"""Get vulnerability timeline - SUPPORTS MULTIPLE YEARS"""
print(f"[Vulnerabilities Over Time] Requested data for {years} years")

# Validate years parameter
years = max(1, min(5, int(years)))  # Limit between 1 and 5 years
```

```
from services.data.data_processor import historical_loader

# Get years to load based on parameter - THE KEY FIX!
# REMOVED: current_year = datetime.now().year
# REMOVED: years_to_load = [current_year]  # CHANGED: Always only current
year!

# NEW CODE: Respect the years parameter
current_year = datetime.now().year
years_to_load = [current_year - i for i in range(years)]

print(f"[Vulnerabilities Over Time] Loading years: {years_to_load}")

monthly_counts = defaultdict(int)
total_cves = 0

for year in years_to_load:
year_cves = historical_loader.get_year_data(year)
print(f"[Vulnerabilities Over Time] Processing {year}: {len(year_cves)}
CVEs")
total_cves += len(year_cves)

for cve in year_cves:
pub_date = cve.get('Published', '')
if pub_date:
try:
if 'T' in pub_date:
dt = datetime.fromisoformat(pub_date.replace('Z', '+00:00'))
else:
dt = datetime.strptime(pub_date[:10], '%Y-%m-%d')
month_key = dt.strftime('%Y-%m')
monthly_counts[month_key] += 1
except Exception as e:
continue

# Clear cache after processing to save memory
if str(year) in historical_loader._year_cache:
print(f"[Vulnerabilities Over Time] Clearing cache for {year} to save
memory")
del historical_loader._year_cache[str(year)]

sorted_months = sorted(monthly_counts.items())

result = {
'labels': [item[0] for item in sorted_months],
'values': [item[1] for item in sorted_months],
'total_cves': total_cves,
'months_covered': len(sorted_months),
'raw_data': dict(monthly_counts),
'years_included': years_to_load  # Added this to track which years are
included
}

print(f"[Vulnerabilities Over Time] Generated timeline:
{result['total_cves']} CVEs across {result['months_covered']} months")

return result

def get_filtered_historical_cves(year: int = None, month: int = None, day:
int = None) -> List[Dict[str, Any]]:
"""Get filtered CVEs from historical data - LAZY LOADING - MEMORY
OPTIMIZED"""
print(f"[Historical Filter] Getting filtered data for year={year},
month={month}, day={day}")
```

```
# Year is required for historical filtering
if not year:
print("[Historical Filter] No year specified, returning empty list")
return []

# Lazy import to avoid circular dependencies
from services.data.data_processor import historical_loader

# Load specific year data for efficient filtering - LAZY LOAD
year_cves = historical_loader.get_year_data(year)
print(f"[Historical Filter] Loaded {len(year_cves)} CVEs for year {year}")

# If no month specified, return all CVEs for the year
if not month:
print(f"[Historical Filter] No month filter, returning all
{len(year_cves)} CVEs for {year}")
return year_cves

# Apply hierarchical filtering: year ? month ? day
filtered_cves = []
for cve in year_cves:
pub_date = cve.get('Published', '')
if pub_date:
try:
if 'T' in pub_date:
dt = datetime.fromisoformat(pub_date.replace('Z', '+00:00'))
else:
dt = datetime.strptime(pub_date[:10], '%Y-%m-%d')

# CRITICAL: Check BOTH year and month
if dt.year == year and dt.month == month:
# If day is specified, check day too
if day is None or dt.day == day:
filtered_cves.append(cve)
except Exception as e:
continue

print(f"[Historical Filter] Filtered result: {len(filtered_cves)} CVEs for
{year}-{month:02d}" + (f"-{day:02d}" if day else ""))

# Debug: Show sample of filtered results
if filtered_cves:
sample_count = min(3, len(filtered_cves))
print(f"[Historical Filter] Sample of filtered CVEs:")
for i in range(sample_count):
cve = filtered_cves[i]
print(f"  - {cve.get('ID', 'Unknown')}: {cve.get('Published', 'No
date')}")

return filtered_cves
```